SQL Server Developer

Выборки из XML и JSON полей

otus.ru

REC **Проверить, идет ли запись**

Меня хорошо видно && слышно?

Ставим “+”, если все хорошо
“-”, если есть проблемы

XML, JSON в SQL Server

```
После занятия вы сможете
писать выборки из XML полей;
писать выборки из JSON полей.

Краткое содержание
варианты выборок из XML и JSON полей;
написание примеров выборок из разного xml'a.

Домашние задания
XML и JSON
```

Напишите в чат

- ➕ сталкивались с XML/JSON
- ➖ не знаете, что это такое

Активно участвуем

Off-topic обсуждаем в учебной группе telegram

Задаем вопрос в чат или голосом

Вопросы вижу в чате, могу ответить не сразу

Использование

- интеграции с внешними системами
- хранение конфигураций и настроек
- хранение в базе полуструктурированных данных

XML	JSON
eXtensible Markup Language	JavaScript Object Notation
большие объемы, сложные типы	небольшие объемы
генерация документов в "человеческом" виде (XML + XSLT)	хранение настроек

XML http://www.cbr.ru/scripts/XML_daily.asp
JSON https://api.open-meteo.com/v1/forecast?latitude=55.75&longitude=37.62&current=temperature&timezone=Europe/Moscow

XML & JSON

1. иерархия
2. избыточность
3. отсутствие фиксированной схемы
4. гибкость

```xml
<person>
  <firstName>Иван</firstName>
  <lastName>Иванов</lastName>
  <address>
    <streetAddress>Московское ш., 101, кв.101</streetAddress>
    <city>Ленинград</city>
    <postalCode>101101</postalCode>
  </address>
  <phoneNumbers>
    <phoneNumber>812 123-1234</phoneNumber>
    <phoneNumber>916 123-4567</phoneNumber>
  </phoneNumbers>
</person>
```

```json
{
    "firstName": "Иван",
    "lastName": "Иванов",
    "address": {
        "streetAddress": "Московское ш., 101, кв.101",
        "city": "Ленинград",
        "postalCode": "101101"
    },
    "phoneNumbers": [
        "812 123-1234",
        "916 123-4567"
    ]
}
```

XML: Структура

- 1я строка - версия и кодировка, если не указана default = UTF-8
- корневой элемент - обязателен
- теги должны быть закрыты: `<Item></Item>`, `<Item/>`
- корректная вложенность тэгов: `<Order><Item></Order></Item>` (err!)
- тэги регистрозависимы: `<order></Order>` (err!)
- экранирование спецсимволов:
  - `< => &lth;`
  - `> => &gth;`
  - `& => &`

XML

```xml
<!--    теги в угловых скобках
        иерархия через вложенность элементов
        нет понятия массивов (только повторяющиеся элементы)
-->
<?xml version="1.0" encoding="UTF-8"?>
<Orders>
    <Order>
        <Number>S043659</Number>
        <Date>2011-05-31T00:00:00</Date>
        <Account>Microsoft</Account>
        <Item>
            <Price>59.99</Price>
            <Quantity>1</Quantity>
        </Item>
    </Order>
    <Order>
        <Number>S043661</Number>
        <Date>2011-06-01T00:00:00</Date>
        <Account>Nokia</Account>
        <Item>
            <Price>24.99</Price>
            <Quantity>3</Quantity>
        </Item>
    </Order>
</Orders>
```

JSON: Структура

- `{}` - объекты
- `[]` - массивы, доступ к элементу по номеру
  - нумерация с 0
- поддержка простых типов: число, строка, boolean

```
/*  фигурные скобки для объектов
    ключи всегда в кавычках
    массивы в квадратных скобках*/
{
    "Orders": [
        {
            "Order": {
                "Number": "S043659",
                "Date": "2011-05-31T00:00:00"
            },
            "Account": "Microsoft",
            "Item": {
                "Price": 59.99,
                "Quantity": 1
            }
        },
        {
            "Order": {
                "Number": "S043661",
                "Date": "2011-06-01T00:00:00"
            },
            "Account": "Nokia",
            "Item": {
                "Price": 24.99,
                "Quantity": 3
            }
        }
    ]
}
```

Выгрузка и загрузка (JSON, для XML - аналогично)

можем преобразовать в таблицу и обратно

Built-in functions
ISJSON
JSON_VALUE
JSON_QUERY

OPENJSON

Transforms JSON text to table

```
[
  {
    "Number":"SO43659",
    "Date":"2011-05-31T00:00:00"
    "AccountNumber":"AW29825",
    "Price":59.99,
    "Quantity":1
  },
  {
    "Number":"SO43661",
     "Date":"2011-06-01T00:00:00"
      "AccountNumber":"AW73565",
      "Price":24.99,
       "Quantity":3
  }
]
```

Number	Date	Customer	Price	Quantity
SO43659	2011-05-31T00:00:00	MSFT	59.99	1
SO43661	2011-06-01T00:00:00	Nokia	24.99	3

FOR JSON

Formats result set as JSON text.

Table 2 XML/JSON

```
SELECT …
FROM …
FOR XML PATH | AUTO | RAW | EXPLICIT
FOR JSON PATH | AUTO [WITHOUT_ARRAY_WRAPPER, INCLUDE_NULL_VALUES]
```

- PATH Создает структуру на основе псевдонимов колонок
- AUTO Создает структуру на основе иерархии таблиц (join)
- XML
  - RAW Каждая строка - элемент с атрибутами или вложенными элементами.
  - EXPLICIT `[ElementName!TagNumber!AttributeName!Directive]`
- XML: https://docs.microsoft.com/ru-ru/sql/relational-databases/xml/for-xml-sql-server
- JSON: https://docs.microsoft.com/ru/sql/relational-databases/json/json-data-sql-server

demo: `01-for_xml_json.sql`

XML 2 Table (парсинг)

```sql
-- читаем xml
DECLARE @xml xml
SELECT @xml = BulkColumn
FROM OPENROWSET(BULK N'<путь к файлу>', SINGLE_CLOB) as data

EXEC sp_xml_preparedocument @handle OUTPUT, @xml

-- вариант 1 openXML
SELECT *
FROM OPENXML (@handle,'/Customer/Order')
WITH ( -- навигация
  OrderNumber    int         '../@OrderNumber',   --путь
  CustomerID  varchar(10) '../@CustomerID',
  OrderDate   datetime    '../@OrderDate',
  ProdID      int         '@ProductID',
  Qty         int         'Quantity'
  )

EXEC sp_xml_removedocument @handle

-- вариант 2: через XQuery (более современный): nodes() и value() - см. 04-xml_data_type.sql
```

xPath - язык навигации по XML-документу

- выбор узлов (элементы, атрибуты и др.) по путям и условиям

Выражение XPath	Результат
book	Все элементы “book”
/bookstore	Корневой элемент “bookstore”
/bookstore/book/author	Все author внутри book внутри bookstore
/bookstore//author	Все author внутри любых подэлементов bookstore

```
<?xml version="1.0" encoding="windows-1251"?>
<bookstore>
   <book category="COOKING">
     <title lang="it">Everyday Italian</title>
     <author>Giada De Laurentiis</author>
     <year>2005</year>
     <price>30.00</price>
   </book>
   <book category="CHILDREN">
     <title lang="en">Harry Potter</title>
     <author>J K. Rowling</author>
     <year>2005</year>
     <price>29.99</price>
   </book>
</bookstore>
```

demo: `02-open_xml.sql`

xPath (примеры для самостоятельного изучения)

Выражение XPath	Результат
/bookstore/book[1]	Первая книга
/bookstore/book[last()]	Последняя книга
//book[@category='COOKING']	book с категорией COOKING
//book[year > 2000]	book, где year больше 2000
//book[year > 2000]/title	Выбирает все элементы title элементов book, где year больше 2000

```
<?xml version="1.0" encoding="windows-1251"?>
<bookstore>
   <book category="COOKING">
     <title lang="it">Everyday Italian</title>
     <author>Giada De Laurentiis</author>
     <year>2005</year>
     <price>30.00</price>
   </book>
   <book category="CHILDREN">
     <title lang="en">Harry Potter</title>
     <author>J K. Rowling</author>
     <year>2005</year>
     <price>29.99</price>
   </book>
</bookstore>
```

примеры http://www.zvon.org/xxl/XPathTutorial/General_rus/examples.html

JSON 2 Table (парсинг)

```sql
-- читаем json
DECLARE @json nvarchar(max)
SELECT @json = BulkColumn
FROM OPENROWSET(BULK N'<путь к файлу>', SINGLE_CLOB) as data

SELECT *
FROM OPENJSON (@json, '$.Suppliers')
WITH ( -- навигация
    Id          INT,
    Supplier    NVARCHAR(100)   '$.SupplierInfo.Name',
    Contact     NVARCHAR(MAX)   '$.Contact' AS JSON,
    City        NVARCHAR(100)   '$.CityName'
)
```

Навигация по JSON-документу

Выражение JSON Path	Описание
$	Сам JSON-объект (корень, контекст)
$.property1	Свойство
$.property1[5]	Шестой элемент в массиве
$.property1.property2[5].property3	
lax$.property1	Если путь не найден, то возвращается пустое значение. По умолчанию.
strict$.property1	Если путь не найден, то ошибка

demo: `03-open_json.sql`

Типы данных в SQL Server для XML & JSON

- XML - отдельный тип (не строка!)
- JSON - отдельный тип в `SQL Server 2025`
- отдельный тип = есть набор методов/функций для работы

```sql
-- до 2 ГБ!
declare @xml xml;
set @xml  = '<tag>text</tag>'

create table t1 (col xml) -- хранение в таблице
insert t1 values(@xml)


declare @json nvarchar(max); -- строковый тип
set @json  = N'{}'

create table t2 (col nvarchar(max)) -- хранение в таблице
insert t2 values(@json)
```

Тип данных XML

- хранит валидный xml (можно со схемой)
  - документы или фрагменты
- разрешен узлы в виде простого текста
- разрешены NULL и пустые строки
- разрешены CDATA (Character Data)
  - спец раздел в XML
  - позволяет включать текст с символами вроде `<`, `>` и `&` без их экранирования, обозначая его как "сырые" символьные данные, а не разметку
- специальные индексы
- поддержка xQuery и xPath 2.0
- нет сравнения (кроме NULL)
- нет сортировки (`order by`)
- ...

XML: методы/функции для работы

для информации, посмотрим на примере

- query() – извлечение XML фрагментов из XML документов;
- value() – извлечение значений конкретных узлов или атрибутов XML документов;
- exist() – проверки существования узла или атрибута.
  - возвращает 1, если узел или атрибут найден, и 0, если не найден;
- modify() – изменяет XML документ;
- nodes() – разделяет XML документ на несколько строк по узлам.
  - используется как вход в "XQuery-подзапросах"

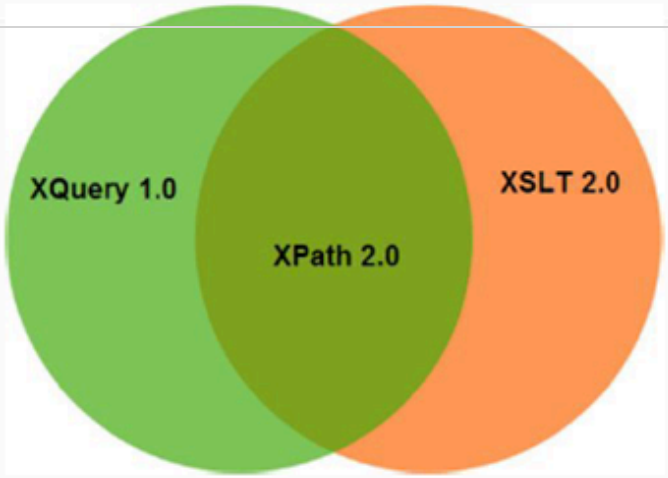
XML: Технологии

набор стандартов для работы с XML-данными

XML-технология	Применение	Использование/пример
XPath *рассмотрели*	Навигация, фильтрация	`//Book[Price > 20]` выбор книг дороже 20 рублей
XQuery	Сложные выборки	`for $x in /Catalog/Book where $x/@id=1 return $x`
XML Schema (XSD)	Обмен между системами	описание структуры и валидация
XSLT	Преобразование	формат для преобразования XML в другие форматы (HTML, JSON, текст)

xQuery

- Аналог SQL для XML
- Типизированный, декларативный, регистрозависимый
- Использует XPath
- Похож на SQL (FLWOR-выражения)
  for let where order by return
  - **for** – задает переменную для цикла
  - **let** – присваивание секвенции
  - **where** – задает фильтр для выбираемых данных
  - **order by** – указывает порядок сортировки
  - **return** – указывает выбираемые значения




```
for $x in doc("books.xml")/bookstore/book
where $x/price>30
order by $x/title
return $x/title
```

- https://ppt-online.org/23233
  (oracle, но хорошие простые примеры XQuery)

XSD (XML Schema Definition)

- схема для валидации XML-документов
- определяет:
  - структуру (какие элементы/атрибуты допустимы)
  - типы данных (число, строка, дата)
  - ограничения (минимальное значение, обязательные поля)

```xml
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="book">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="title" type="xs:string"/>
        <xs:element name="author" type="xs:string"/>
        <xs:element name="price">
          <xs:complexType>
            <xs:simpleContent><xs:extension base="xs:decimal"><xs:attribute name="currency" type="xs:string"/></xs:extension>
            </xs:simpleContent>
          </xs:complexType>
        </xs:element>
      </xs:sequence>
      <xs:attribute name="id" type="xs:integer"/>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

04-xml_data_type.sql

XSLT: "таблица стилей" для XML

- `books.xml` - данные + ссылка на xslt

```
<?xml version="1.0" encoding="UTF-8"?>
<?xml-stylesheet type="text/xsl" href="books.xslt"?>
... данные
```

- `books.xslt` - описание как преобразовываем (для показа пользователю)
- `books.xml` можно просмотреть через `Excel`, указав `books.xslt` как `stylesheet`
  - можно и в браузере, но нужно настраивать политики безопасности

Новые поступления книг

- Властелин колец – Дж. Р. Р. Толкиен, 500 руб.
- Гарри Поттер и Философский камень – Дж. К. Роулинг, 400 руб.

demo `books.xml`, `books.xslt`

JSON: методы/функции для работы

для информации, посмотрим на примере

- isjson()
- json_query()
- json_value()
- json_modify()
- SQL Server 2025 - намного больше...

demo: `05-json_value_query.sql`

Итоги

Характеристика	XML (язык разметки)	JSON
Терминология	элементы, атрибуты, тэги, коментарии...	объекты, массивы
Простейший синтаксис	`<tag>content</tag>` длиннее	`{"key": "value"}` короче
Поддержка типов	только текст (строковый тип)	числа, строки, boolean, null
Валидация	схема XSD	парсинг
Тип в SQL	➕ `XML`	➖ `NVARCHAR(MAX)`, ➕SQL Server 2025
Экранирование спецсимволов	- `< => &lth;` - `> => &gth;` - `& => &`	- обратный слэш: - `{} => \{\}` - `" => \"`
Table 2 XML/JSON	`SELECT * FOR XML...`	`SELECT * FOR JSON...`
XML/JSON 2 Table (парсинг)	- `OPENROWSET` - `OPENXML` - `XQuery`, `XPath`	- `OPENROWSET` - `OPENJSON` - `JSONPath`

Вопросы?

Ставим “+”,
если вопросы есть

Ставим “–”,
если вопросов нет

Заполните, пожалуйста, опрос о занятии по ссылке в чате